Algorithm2: *Problem Solving* with *Algorithms* and *Data Structures*

Lecture 06.

탐욕법

경북대학교 배준현 교수
(joonion@knu.ac.kr)

탐욕적 접근법: *Greedy Approach*

모든 선택지를 고려해 보고 그중 전체 답이 가장 좋은 것을 찾는 것(완전 탐색)이 아니라,
각 단계마다 지금 당장 가장 좋은 해의 일부를 순차적으로 선택하는 전략

탐욕적 알고리즘을 최적화 문제에 적용할 때는 많은 경우 최적해를 찾지 못한다.
따라서, 탐욕적 접근법이 반드시 최적해를 찾을 수 있다는 것을 증명해야 한다.

탐욕적 알고리즘을 사용하는 두 가지 경우:

1. 탐욕법을 사용해도 항상 최적해를 구할 수 있는 문제: 동적계획법보다 더 효율적
2. 최적해가 아니라 근사해를 찾는 경우: NP-Hard 문제 등(예: TSP)에 적용

탐욕적 알고리즘 레시피

1. 문제의 답을 만드는 과정을 여러 조각으로 나눈다.
2. 각 조각마다 어떤 우선순위로 선택할 지 결정한다. (정렬, 우선순위 큐 활용)
3. 두 가지 속성을 증명해 본다.

탐욕적 선택 속성: *greedy choice property*

각 단계에서 선택한 답을 포함하는 최적해가 항상 존재함을 증명한다.

최적 부분 구조: *optimal substructure*

부분 문제의 최적해에서 전체 문제의 최적해를 만들 수 있음을 증명한다.

탐욕적 선택 속성: *greedy choice property*

현재 상황에서 최선의 선택을 해 나가면, 결국 전체 문제에 대한 최적해로 이어질 수 있다.

이 속성을 만족하려면, 현재의 선택으로 인해 이후의 선택들이 최적해에 영향을 주지 않아야 한다.

예) 동전 교환(거스름돈) 문제: *Coin Change Problem*

주어진 동전 집합에서 주어진 금액을 최소 개수의 동전으로 구성하는 문제

그리디 접근법: 매번 현재 금액보다 작은 가장 큰 단위의 동전을 선택한다.

매 순간 최적의 선택을 하지만, 전체 문제에 대한 최적해로 이어지지 않는다.

예) $S = \{1, 3, 4\}, W = 6$: $greedy\ solution = \{4, 1, 1\}, optimal\ solution = \{3, 3\}$

최적 부분 구조: *optimal substructure*

전체 문제의 최적해가 작은 부분 문제들의 최적해로 구성될 수 있다.

이 속성을 만족해야, 각 단계에서 최적의 선택을 함으로써 전체 문제의 최적해를 보장할 수 있다.

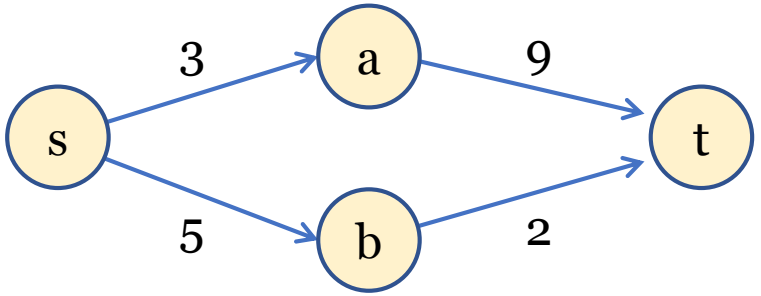
예) 최단 경로 문제: *Shortest Path Problem*

가중치가 있는 방향 그래프에서 출발 정점으로부터 도착 정점으로의 최단 경로 찾기

그리디 접근법: 현재 정점에 연결된 간선 중에서 가장 작은 가중치를 가진 간선을 선택한다.

최적 부분 구조: 현재 선택한 간선은 전체 최단 경로의 일부분이 된다.

최적 부분 구조가
성립하지 않는 반례:



탐욕적 알고리즘의 구조

1. 해결할 문제의 분할:
   문제를 해결하기 위해 해결할 문제의 구성 요소를 여러 단계로 분할한다.

2. 단계별 최적 선택:
   각 단계마다 가장 최적이라고 생각하는 (제약 조건을 고려한) 결정을 내린다.

   (*selection procedure*) 후보 집합에서 솔루션 집합에 들어갈 원소를 선택 (정렬, 우선순위 큐)

   (*feasibility check*) 새로운 원소를 추가한 솔루션 집합이 제약 조건에 맞는지 확인

   (*solution check*) 새로운 원소를 추가한 솔루션 집합이 최종 솔루션인지 확인

3. 최종 해결책 도출:
   각 단계에서 내린 결정을 종합하여 최종 해결책을 구한다.

탐욕적 접근법을 사용할 수 있는 대표적인 문제들:

1. **거스름돈 문제**
2. **분할 가능한 배낭 문제**
3. **활동 선택 문제**: activity selection (**회의실 배정**)
4. **스케줄링 문제: 데드라인이 있는 스케줄 짜기**
5. **최단 경로 문제**: Dijkstra algorithm
6. **최소 신장 트리**(MST) **문제**: Prim algorithm & Kruskal algorithm
7. **허프만 알고리즘**: Huffman algorithm

AL2.6.1: 출전 순서 정하기 (ID: MATCHORDER)

문제

전세계 최대의 프로그래밍 대회 알고스팟 컵의 결승전이 이틀 앞으로 다가왔습니다. 각 팀은 n명씩의 프로 코더들로 구성되어 있으며, 결승전에서는 각 선수가 한 번씩 출전해 1:1 경기를 벌여 더 많은 승리를 가져가는 팀이 최종적으로 우승하게 됩니다. 각 팀의 감독은 대회 전날, 주최측에 각 선수를 출전시킬 순서를 알려 주어야 합니다.

결승전 이틀 전, 한국팀의 유감독은 첩보를 통해 상대 러시아팀의 출전 순서를 알아냈습니다. 이 대회에서는 각 선수의 실력을 레이팅(rating)으로 표현합니다. 문제를 간단히 하기 위해 1:1 승부에서는 항상 레이팅이 더 높은 선수가 승리하고, 레이팅이 같을 경우 우리 선수가 승리한다고 가정합시다.

경기	1	2	3	4	5	6
러시아팀	3,000	2,700	2,800	2,200	2,500	1,900
한국팀	2,800	2,750	2,995	1,800	2,600	2,000

표와 같이 출전 순서를 정했다고 하면 한국팀은 2번, 3번, 5번, 6번 경기에서 승리해 전체 네 경기를 이기게 됩니다. 그러나 대신 4번 경기와 1번 경기에 나갈 선수를 바꾸면 1번 경기만을 제외하고 모든 경기에 승리할 수 있지요. 상대방 팀 선수들의 순서를 알고 있을 때, 어느 순서대로 선수들을 내보내야 승수를 최대화할 수 있을까요?

Lec. 06. 탐욕법

입력 입력의 첫 줄에는 테스트 케이스의 수 C (C≤50)가 주어집니다. 각 테스트 케이스의 첫 줄에는 각 팀 선수의 수 N(1≤N≤100)가 주어집니다. 그 다음 줄에는 N개의 정수로 러시아팀 각 선수의 레이팅이 출전 순서대로 주어지며, 그 다음 줄에는 N개의 정수로 한국팀 각 선수의 레이팅이 무순으로 주어집니다. 모든 레이팅은 1 이상 4000 이하의 정수입니다.

출력 각 테스트 케이스마다 한 줄에 한국팀이 얻을 수 있는 최대 승수를 출력합니다.

예제 입력

```
3
6
3000 2700 2800 2200 2500 1900
2800 2750 2995 1800 2600 2000
3
1 2 3
3 2 1
4
2 3 4 5
1 2 3 4
```

예제 출력

```
5
3
3
```

무식하게 풀 수 있을까?

모든 경우의 수 탐색: n명의 선수가 있으면 $n!$개의 순열을 탐색해야 하므로,

n이 조금만 커져도 시간 초과가 될 것이다.

완전 탐색 알고리즘에 메모이제이션을 적용하는 것은 어떨까?

$taken$: 각 한국팀 선수를 이미 순서에 추가했는지 여부를 나타내는 불린 값 배열

$order(taken)$: 남은 선수들을 적절히 배치해서 얻을 수 있는 최대 승수

SST의 노드 개수 $O(2^n)$, $taken$에서 승수 계산 $O(n)$: 시간 복잡도는 $O(n2^n)$

탐욕적으로 접근할 수 있을까?

출전할 한국 선수를 정하는 전략: 상대방 선수를 한 명씩 골라서

1. 상대방 선수를 이길 수 있는 선수가 있으면,
   그 중 레이팅이 가장 낮은 선수를 상대방 선수와 경기시킨다.
2. 상대방 선수를 이길 수 있는 선수가 없다면,
   남은 선수 중 가장 레이팅이 낮은 선수를 상대방 선수와 경기시킨다.

위 탐욕적 전략은 한 번에 한 명씩 가장 최선의 선수를 선택하는 전략을 취한다.

이 전략은 탐욕적 선택 속성을 만족하는가?

이 전략은 최적 부분 구조 속성을 만족하는가?

탐욕적 선택 속성의 증명

항상 우리가 하는 선택을 포함하는 최적해가 존재하는가?

한국팀이 질 수밖에 없는 경우: 상대팀 선수가 모든 우리 팀 선수보다 레이팅이 높을 경우

가장 레이팅이 낮은 선수 A 대신 선수 B를 내보내는 최적해가 있다고 가정하자.

러시아	...	999,999,999	...	x	...	1,900
한국	...	B	...	A	...	2,000

최적해에서 이 두 선수의 순서를 바꾸면 어떻게 될까?

상대 선수 x는 레이팅이 더 높은 선수를 상대해야 한다.

따라서, 승수가 더 줄어들 일은 없고, 이 경기에 A를 내보내는 최적해가 존재한다.

한국팀이 이길 수 있는 경우: 상대팀 선수보다 레이팅이 높거나 같은 우리 선수가 있을 경우

승리할 수 있는 가장 레이팅이 낮은 A 대신 레이팅이 더 높은 B를 내보내는 최적해가 있다고 가정

러시아	...	2,000	...	x	...	1,900
한국	...	B	...	A	...	2,000

최적해에서 이 두 선수의 순서를 바꾸면 어떻게 될까?

A를 상대했던 x는 레이팅이 더 높은 선수를 상대해야 하므로 이 순서 또한 최적해가 된다.

승리할 수 있는 가장 레이팅이 낮은 A 대신 레이팅이 더 낮은 C를 내보내는 최적해가 있다고 가정

이 경기는 이길 수 있는 경기이지만 지게 되고, 최적해에서 A와 C의 위치를 바꾸면 이길 수 있다.

승수가 1 늘고 1 줄었으니 결과적으로 여전히 최적해가 된다.

최적 부분 구조의 증명

항상 최적의 선택만을 내려서 전체 문제의 최적해를 얻을 수 있는가?

첫 번째 경기에 나갈 선수를 선택하고 나면, 남은 선수들을 경기에 배정하는 부분 문제를 얻을 수 있다.
남은 부분 문제에서 최다승을 얻는 것이 전체 문제에서 최다승을 얻는 최적해의 일부분이다.

각 부분 문제는 전체 문제와 동일한 형태를 가진다.
각 부분 문제의 최적해를 결합하면 전체 문제의 최적해를 구성할 수 있다.

※ 동적 계획법을 적용할 때에도 반드시 최적 부분 구조 속성을 증명해야 한다.
다행히, 최적 부분 구조는 대개 매우 자명해서 따로 증명할 필요가 없는 경우가 대부분이다.

```
'''
P.6.1. 출전 선수 정하기 (ID: MATCHORDER)
'''
for _ in range(int(input())):
    n = int(input()) # 각 팀 선수의 수
    russian = list(map(int, input().split())) # 러시아 선수의 레이팅 (출전 순서)
    korean = list(map(int, input().split())) # 한국 선수의 레이팅 (무순)
    print(matchorder(n, russian, korean))
```

```python
from bisect import bisect_left

# 출전 순서 정하기 문제를 해결하는 탐욕적 알고리즘
def matchorder(n, russian, korean):
    korean.sort() # 한국 선수의 레이팅을 오름차순 정렬
    wins = 0
    for i in range(n):
        if korean[-1] < russian[i]: # 가장 레이팅이 높은 한국 선수가 이길 수 없는 경우
            korean.pop(0) # 가장 레이팅이 낮은 선수를 출전시킨다.
        else: # 이 외의 경우
            wins += 1 # 이길수 있는 선수 중 가장 레이팅이 낮은 선수를 출전시킨다.
            korean.pop(bisect_left(korean, russian[i]))
    return wins
```

이진 탐색: $O(\lg N)$

AL2.6.2: 문자열 합치기 (ID: STRJOIN)

문제

프로그래밍 언어 C 의 큰 문제점 중 하나는 언어 차원에서 문자열 변수형을 지원하지 않는다는 것입니다. C 에서는 문자 배열로 문자열을 표현하되 \0 (NULL) 로 문자열의 끝을 지정하는데, 이래서는 문자열의 길이를 쉽게 알 수 있는 방법이 없기 때문에 여러 가지 문제가 발생하게 됩니다.

```
void strcat(char* dest, const char* src) {
 // dest 의 마지막 위치를 찾는다
 while(*dest) ++dest;
 // src 를 한 글자씩 dest 에 옮겨 붙인다
 while(*src) *(dest++) = *(src++);
 // 문자열의 끝을 알리는 \0 을 추가한다
 *dest = 0;
}
```

이런 문제 중 하나로 문자열을 조작하는 함수들의 동작 시간이 불필요하게 커진다는 것이 있습니다. 앞에 주어진 함수 strcat() 은 문자열 dest 뒤에 src 를 붙이는 함수인데, 실행 과정에서 반복문을 두 문자열의 길이를 합한 만큼 수행해야 합니다. 이 함수를 사용해 두 개의 문자열을 합치는 비용은 두 문자열의 길이의 합이라고 합시다.

이 함수를 이용해 n 개의 문자열을 순서와 상관없이 합쳐서 한 개의 문자열로 만들고 싶습니다. 순서가 상관 없다는 말은 {al,go,spot} 을 spotalgo 로 합치든 alspotgo 로 합치든 상관 없다는 의미입니다. 그러나 문자열을 합치는 순서에 따라 전체 비용이 달라질 수 있습니다. 예를 들어 먼저 al 과 go 를 합치고 (2+2=4), 이것을 spot 과 합치면 (4+4=8) 총 12 의 비용이 들지만 al 과 spot 을 합치고 (2+4=6) 이것을 다시 go 에 합치면 (6+2=8) 총 14 의 비용이 필요합니다.

n 개의 문자열들의 길이가 주어질 때 필요한 최소 비용을 찾는 프로그램을 작성하세요.

입력 입력의 첫 줄에는 테스트 케이스의 수 c (c <= 50) 가 주어집니다. 각 테스트 케이스의 첫 줄에는 문자열의 수 n (1 <= n <= 100) 이 주어지며, 다음 줄에는 n 개의 정수로 각 문자열의 길이가 주어집니다. 각 문자열의 길이는 1,000 이하의 자연수입니다.

출력 각 테스트 케이스마다 한 줄에 모든 문자열을 합칠 때 필요한 최소 비용을 출력합니다.

예제 입력

```
3
3
2 2 4
5
3 1 3 4 1
8
1 1 1 1 1 1 1 2
```

예제 출력

```
12
26
27
```

무식하게 풀 수 있을까?

모든 경우의 수: n개의 문자열을 합치는 경우의 수는 $(n-1)!$

동적 계획법으로 풀 경우:

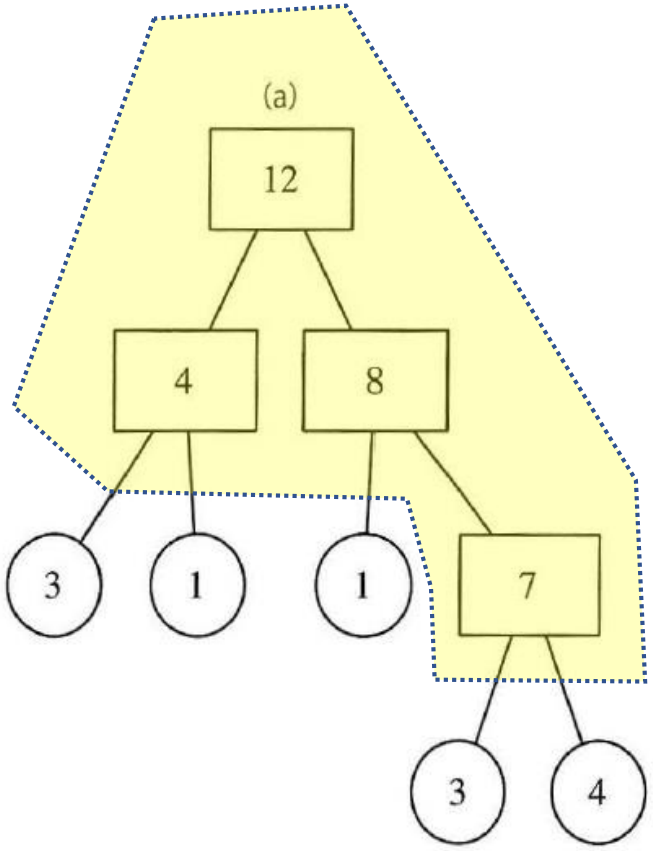
i에서 j까지의 문자열을 합치는 최소 비용은 중간 문자열 k를 기준으로 모든 k에 대해서

i에서 k까지의 비용과 $k+1$에서 j까지의 비용과 이 둘을 합치는 비용의 합 중에서 최솟값이다.

점화식: $dp[i][j] = \min_{i \le k < j}(dp[i][k] + dp[k+1][j] + cost(i,j))$

시간 복잡도: $O(n^3)$

문자열을 합치는 데 드는 비용의 이해: 그림으로 그려보기



리프 노드: 각 문자열의 길이

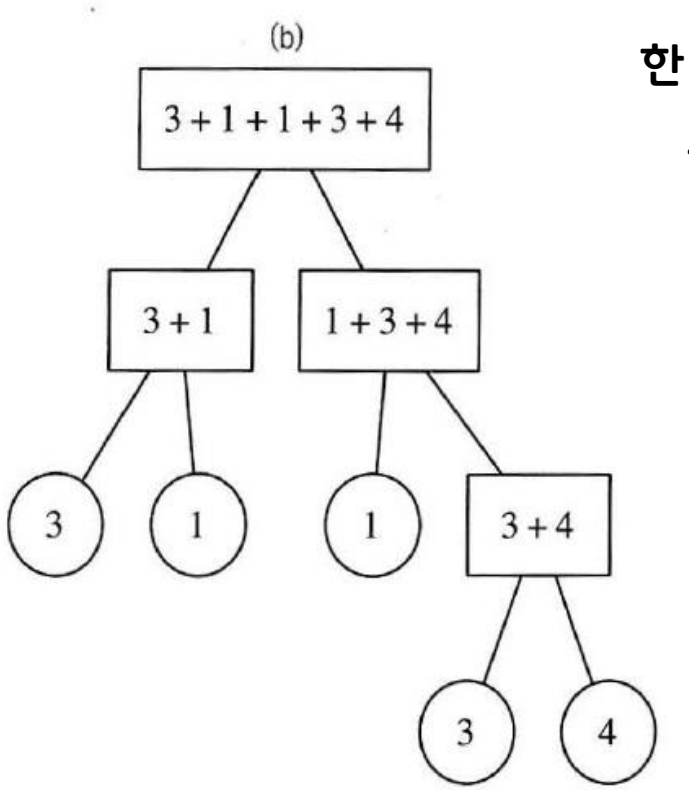
중간 노드: 문자열을 합치는 비용

문자열을 합치는 데 드는 총 비용:

중간 노드에 있는 비용의 합

예) $7 + 4 + 8 + 12 = 31$

탐욕법으로 풀 수 있을까?



한 문자열이 전체 비용에 미치는 영향:

문자열을 병합할 때마다 병합되는 문자열들의 총 길이가 전체 비용에 더해진다.

한 문자열로 인해 발생하는 총 비용: (병합되는 횟수) × (문자열의 길이)

문자열이 길면 길수록 트리의 윗부분에 가깝고,

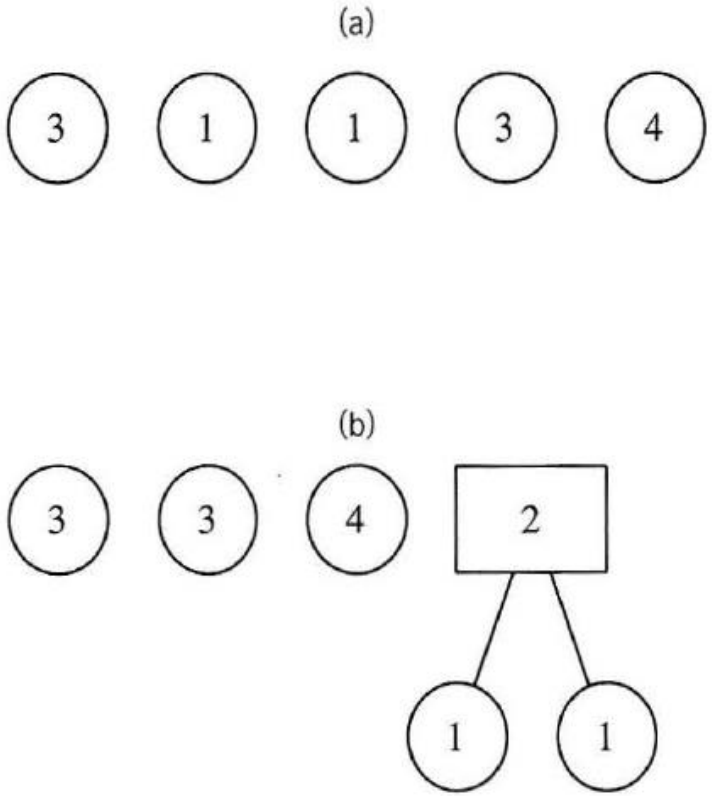
짧으면 짧을수록 아래쪽으로 내려가야 한다.

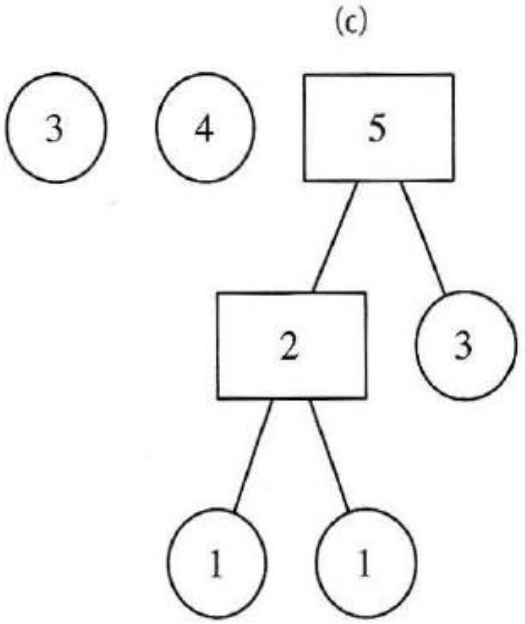
탐욕적 알고리즘의 설계

1. 매 단계에서 두 개의 문자열을 합친다.

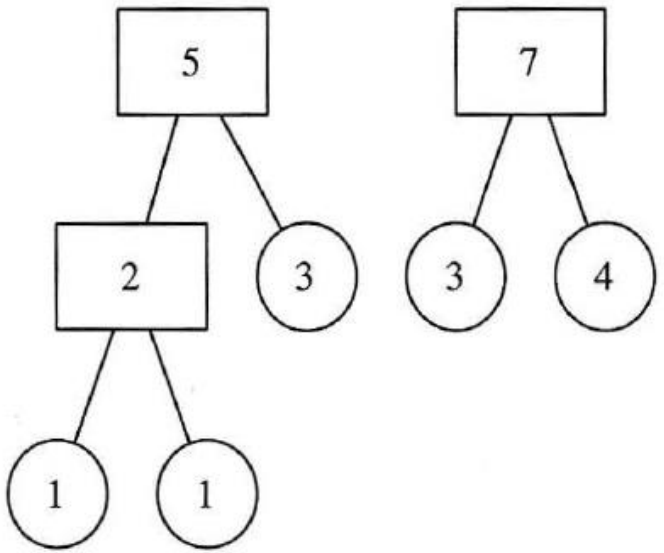
2. 현재 문자열 목록에서 두 개를 골라내어 이들을 합친 뒤,
   합친 문자열을 다시 문자열 목록에 추가하는 과정을 반복한다. (bottom-up)

3. 문자열이 길면 길수록 트리의 위쪽에 있는 것이 유리하므로
   항상 가장 짧은 두 개의 문자열을 합친다.

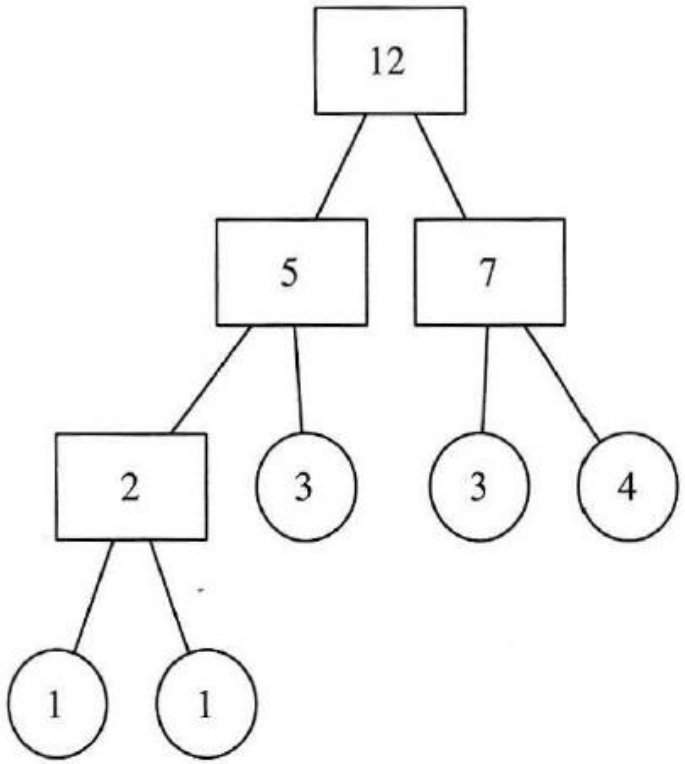

(a)
3
1
1
3
4
(b)
3
3
4
2
1
1


(c)
3
4
5
2
3
1
1

(d)



(e)



우선순위 큐의 활용

우선순위 큐: 가장 작은 원소를 찾거나 새 원소를 추가하는 작업을 모두 $O(logn)$에 가능

1. 모든 문자열을 우선순위 큐에 추가한다.
2. 아래 과정을 $n-1$번(또는 큐의 크기가 1이 될 때까지) 반복한다.
   a. 가장 짧은 두 개의 문자열을 우선순위 큐에서 꺼낸다.
   b. 두 개의 문자열을 합치고 합친 문자열을 다시 우선순위 큐에 추가한다.
   c. 전체 비용에 문자열을 합친 비용을 더한다.
3. 전체 비용을 반환한다.

```
'''
P.6.2. 문자열 합치기 (ID: STRJOIN)
'''
for _ in range(int(input())):
    n = int(input()) # 문자열의 수
    strlen = list(map(int, input().split())) # 문자열의 길이
    print(strjoin(strlen))
```

```python
from heapq import heappush, heappop, heapify

# 문자열 합치기 문제를 해결하는 탐욕적 알고리즘
def strjoin(strlen):
    heapify(strlen) # strlen을 최소힙(우선순위 큐)으로 구성
    ret = 0
    while len(strlen) > 1: # 원소가 하나 남을 때까지
        # 가장 짧은 문자열 두 개를 찾아서 합치고 우선순위 큐에 다시 넣는다.
        len1 = heappop(strlen)
        len2 = heappop(strlen)
        heappush(strlen, len1 + len2)
        ret += len1 + len2
    return ret
```

시간 복잡도: $O(nlogn)$

이론적 배경: *Huffman Algorithm* for *Huffman Code*

허프만 알고리즘: 가변 길이 인코딩/디코딩을 위한 최적 이진 트리(허프만 트리)를 만드는 알고리즘

가변 길이 인코딩: *variable-length encoding*

원문의 각 글자를 서로 길이가 다를 수 있는 비트 패턴으로 바꿔서 원문의 길이를 줄이는(압축) 방법

최적 바이너리 코드 문제: *optimal binary code problem*

원문에 출현하는 글자들의 출현 빈도가 주어질 때, 예상 길이를 최소화하는 비트 패턴을 만든다.

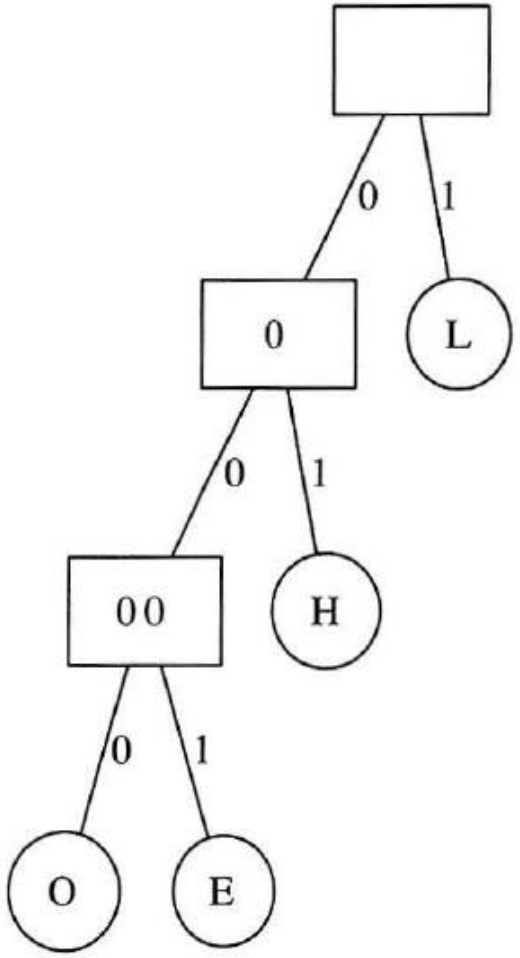


그림 10.5 허프만 코드

H	E	L	L	O
↕	↕	↕	↕	↕
01	001	1	1	000

자주 출현하는 글자는 더 짧은 패턴으로: 이진 트리의 위쪽으로

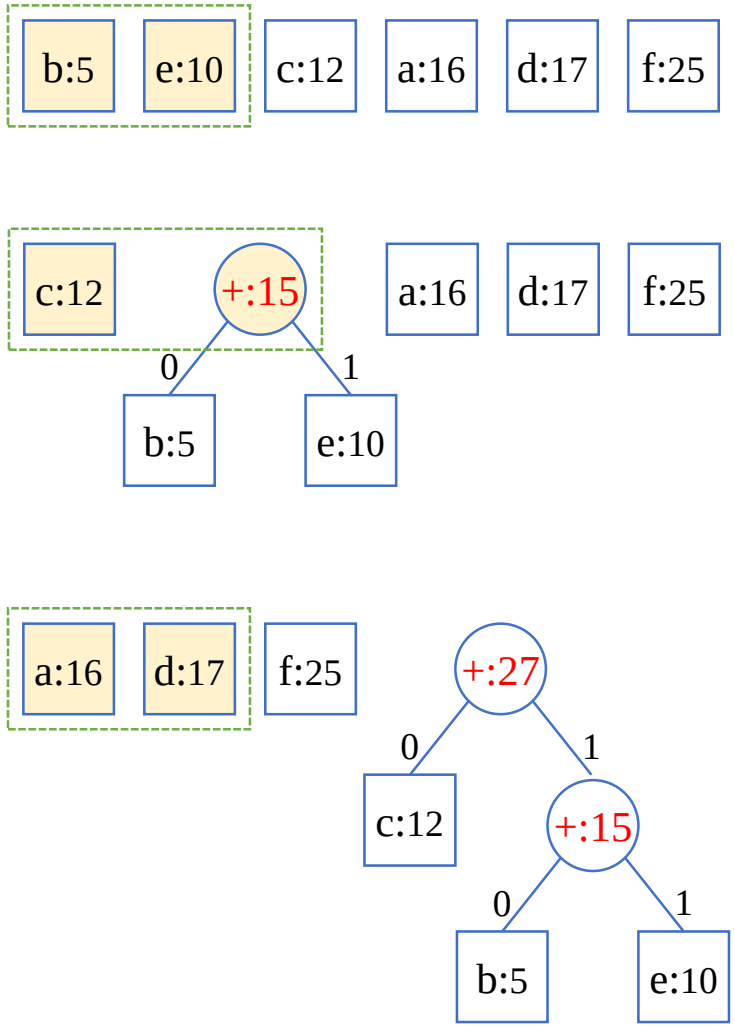
가끔 출현하는 글자는 더 긴 패턴으로: 이진 트리의 아래쪽으로

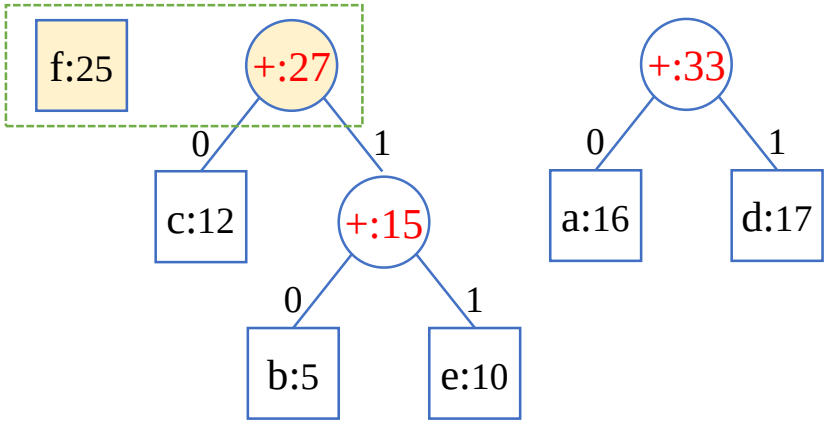
접두어 코드: *prefix code (prefix-free code)*

어떤 글자의 codeword도 다른 codeword의 접두어가 되지 않는다.

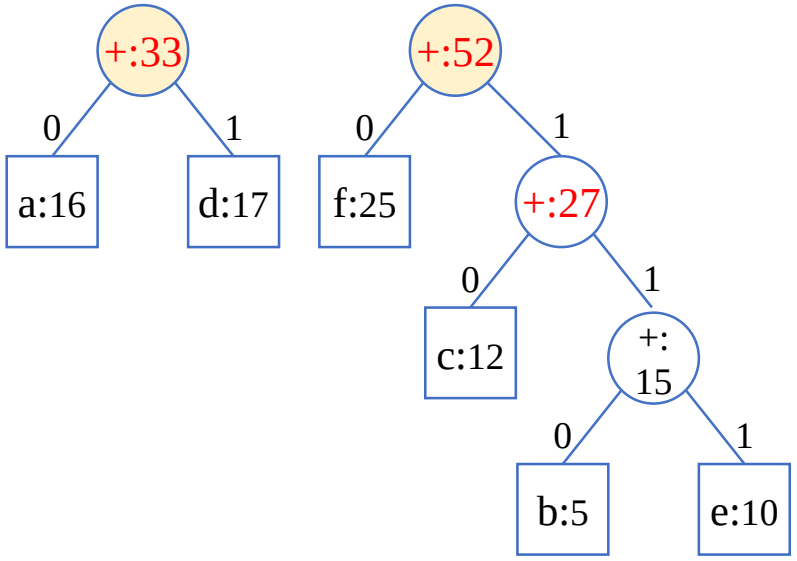
문자열을 디코딩할 때 뒤에 올 글자를 미리 들여다볼(lookahead) 필요가 없다.

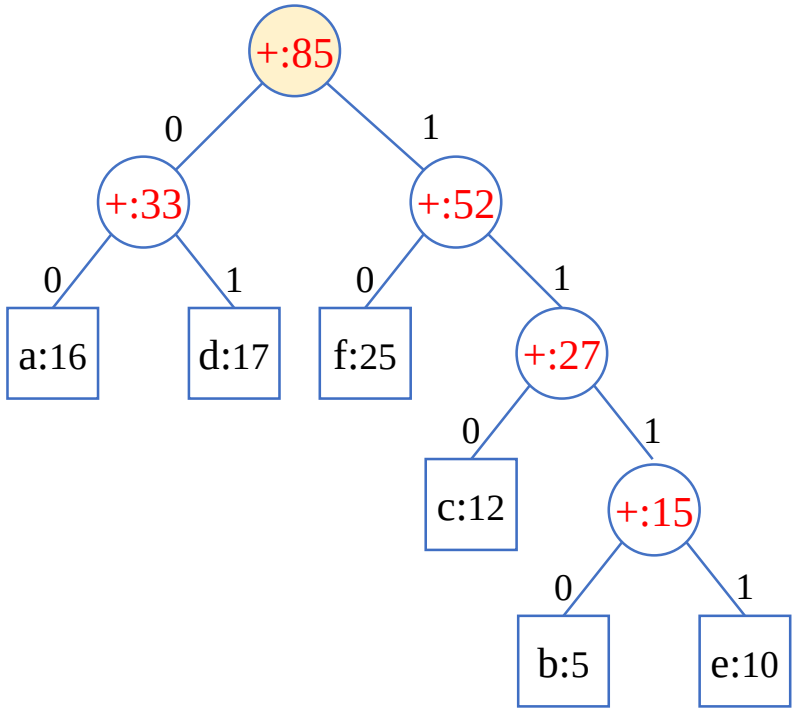
Character	Frequency
a	16
b	5
c	12
d	17
e	10
f	25




f:25
+:27
0
1
c:12
+:15
0
1
b:5
e:10

+:33
0
1
a:16
d:17


+:33
0
1
a:16
d:17
+:52
0
1
f:25
+:27
0
1
c:12
+:
15
0
1
b:5
e:10


+:85
0
1
+:33
+:52
0
1
0
1
a:16
d:17
f:25
+:27
0
1
c:12
+:15
0
1
b:5
e:10

```
n = int(input())
pairs = [input().split() for _ in range(n)]
root = huffman(pairs)
print_preorder(root)
print()
print_inorder(root)
print()
encoder = {}
make_encoder(root, "", encoder)
encoded = encode("abcdef", encoder)
print(encoded)
decoded = []
decode(root, encoded, 0, decoded)
print(*decoded)
```

```
6
a 16
b 5
c 12
d 17
e 10
f 25
```

```
(+, 85) (+, 33) (a, 16) (d, 17) (+, 52) (f, 25) (+, 27) (c, 12) (+, 15) (b, 5) (e, 10)
(a, 16) (+, 33) (d, 17) (+, 85) (f, 25) (+, 52) (c, 12) (+, 27) (b, 5) (+, 15) (e, 10)
0011101100111110
a b c d e f
```

```
class Node:

    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

def print_preorder(node):
    if node:
        print(f"({node.char}, {node.freq})", end = " ")
        print_preorder(node.left)
        print_preorder(node.right)

def print_inorder(node):
    if node:
        print_inorder(node.left)
        print(f"({node.char}, {node.freq})", end = " ")
        print_inorder(node.right)
```

```
def huffman(pairs):
    pq = [(int(f), Node(c, int(f))) for c, f in pairs]
    heapify(pq)
    while len(pq) > 1:
        freq1, node1 = heappop(pq)
        freq2, node2 = heappop(pq)
        node = Node("+", freq1 + freq2)
        node.left = node1
        node.right = node2
        heappush(pq, (freq1 + freq2, node))
    return pq[0][1]
```

```
def make_encoder(node, code, encoder):
    if node.char != '+':
        encoder[node.char] = code
    else:
        make_encoder(node.left, code + "0", encoder)
        make_encoder(node.right, code + "1", encoder)

def encode(s, encoder):
    encoded = ""
    for char in s:
        encoded += encoder[char]
    return encoded
```

```
def decode(node, s, i, decoded):
    global root
    if node.char != '+':
        decoded += [node.char]
        if i < len(s):
            decode(root, s, i, decoded)
    elif s[i] == "0":
        decode(node.left, s, i + 1, decoded)
    else:
        decode(node.right, s, i + 1, decoded)
```

AL2.6.3: 근거리 네트워크 (ID: LAN)

문제

근거리 네트워크(LAN, Local Area Network)는 가정, 학교, 혹은 회사 등의 제한된 공간 내의 컴퓨터들을 연결하는 네트워크입니다. 알고스팟 대학교에서는 캠퍼스의 일부 건물들만이 서로 근거리 네트워크로 연결되어 있었는데, 이번에 캠퍼스 정보화 사업의 일환으로 모든 건물을 모두 연결하려고 합니다. 모든 건물이 서로 연결되어 있다는 것은 건물 사이의 케이블을 이용해 모든 건물 간에 서로 직접/간접적으로 데이터를 주고받을 수 있다는 것을 의미합니다.

문제를 단순화하기 위해, 모든 건물들은 2차원 평면 위에 있는 점이라고 가정합시다. 두 건물을 직접 연결하려면 두 건물 사이의 거리 만큼의 케이블이 필요합니다. 케이블은 항상 두 건물만을 연결할 수 있으며, 두 케이블이 교차한다고 해서 두 케이블이 직접 연결된 것은 아닙니다.

각 건물들의 위치와 이미 설치된 케이블들에 대한 정보가 주어질 때, 모든 건물을 연결하기 위해 추가로 설치해야 할 케이블 길이의 최소 합을 계산하는 프로그램을 작성하세요.

입력 입력의 첫 줄에는 테스트 케이스의 수 C (C <= 50) 가 주어집니다. 각 테스트 케이스의 첫 줄에는 건물의 수 N (N <= 500) 과 이미 설치된 케이블의 수 M (M <= 2000) 이 주어집니다. 각 건물은 0번부터 N-1 번까지의 고유 번호로 구분합니다. 다음 줄에는 N 개의 정수로 0번부터 N-1 번까지 각 건물의 x 좌표가, 그 다음 줄에는 N 개의 정수로 y 좌표가 주어집니다. 각 좌표의 절대값은 1000 이하입니다. 그 후 M 줄에 각 2개의 정수로 각 케이블이 연결하는 두 건물의 번호가 주어집니다. 한 쌍의 건물을 연결하는 케이블이 두 개 이상 있을 수도 있습니다.

출력 각 테스트 케이스마다 한 줄에 추가로 설치해야 할 케이블의 총 길이를 출력합니다. 10^{-7} 이하의 상대/절대 오차가 있는 답도 정답으로 인정합니다.

예제 입력

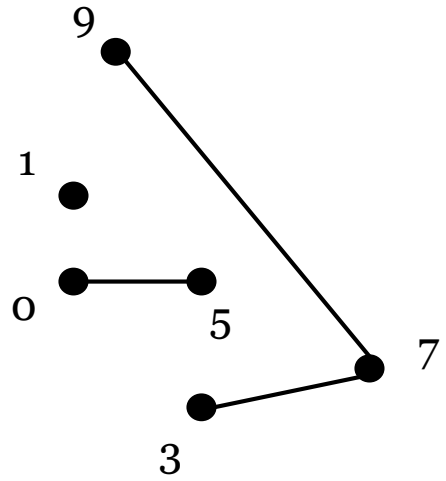
```
2
3 1
0 0 1
0 1 2
0 1
10 5
-7 -7 10 -4 10 -4 -5 0 -10 -6
6 8 -5 3 -4 6 -10 4 -7 10
9 7
7 3
9 7
5 0
8 6
```

예제 출력

```
1.414214
29.704220
```

근거리 네트워크 문제의 이해

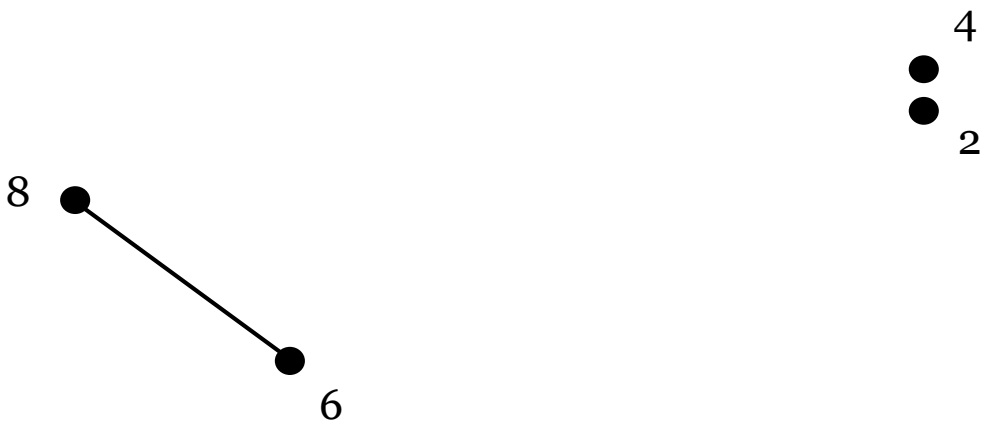
```
10 5
-7 -7 10 -4 10 -4  -5 0 -10 -6
 6  8 -5  3 -4  6 -10 4  -7 10
9 7
7 3
9 7
5 0
8 6
```



모든 건물을 연결해야 한다: 스패닝 트리

추가로 설치할 케이블 길이의 최소 합: MST

가중치: 두 건물사이의 거리 (유클리드 거리)



최소 스패닝 트리: MST, *Minimum Spanning Tree*

가중치가 있는 무방향 연결 그래프에서 가중치의 합이 가장 작은 스패닝 트리를 찾는 문제

그래프: $G = (V, E)$, a *connected*, *undirected*, and *weighted* graph.

스패닝 트리: $T = (V, F)$, where $F \subseteq E$ and T is *connected* and *acyclic*.

최소 스패닝 트리: Find $T = (V, F)$ such that $\sum_{e \in F} w(e)$ is *minimized*.

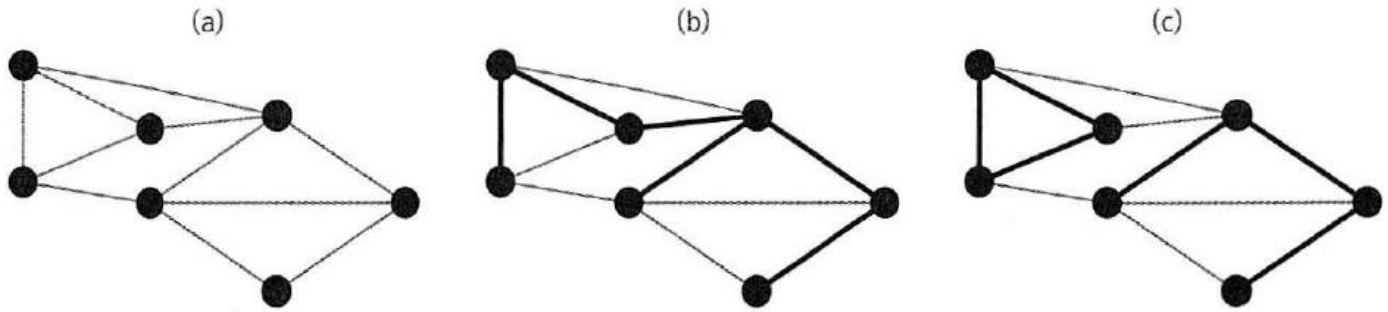


그림 31.1 한 그래프의 올바른 스패닝 트리와 그렇지 않은 스패닝 트리

MST를 찾는 두 가지 탐욕적 알고리즘:

Prim's 알고리즘:

출발 정점에서 시작하여 현재까지 선택된 정점들과 가장 가까운 정점을 선택하여 트리에 추가

시간 복잡도: 일반적으로 $O(|V|^2)$, 우선순위 큐(피보나치 힙)를 사용하면 $O(|E| + |V| \log |V|)$

Dense Graph에 효율적이고, Dijkstra 알고리즘의 구조와 거의 유사함

Kruskal's 알고리즘:

가중치가 가장 작은 간선을 선택해서 해당 간선이 사이클을 만들지 않으면 트리에 추가

시간 복잡도: 간선의 정렬 시간 $O(|E| \log |E|)$ + 유니온-파인드 $O(|V| \log |V|)$

Sparse Graph에 효율적이고, 유니온-파인드 알고리즘에 의존적 (10강(교재 31장)에서 다룰 예정)

Prim's 알고리즘

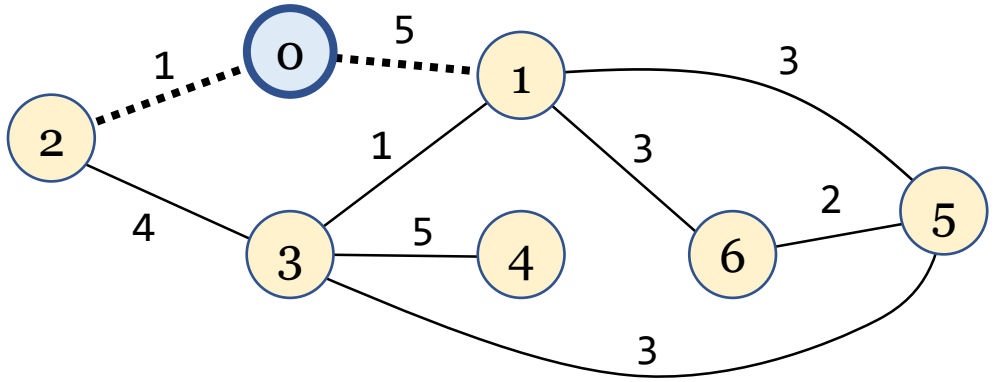
스패닝 트리에 속하지 않은 각 정점에 대해,

스패닝 트리와 이 정점을 연결하는 가장 짧은 간선에 대한 정보를 저장하고,

각 정점을 순회하면서 다음에 추가할 정점을 찾는다.

$minweight[v]$: 스패닝 트리와 정점 v를 연결하는 간선의 최소 가중치를 저장

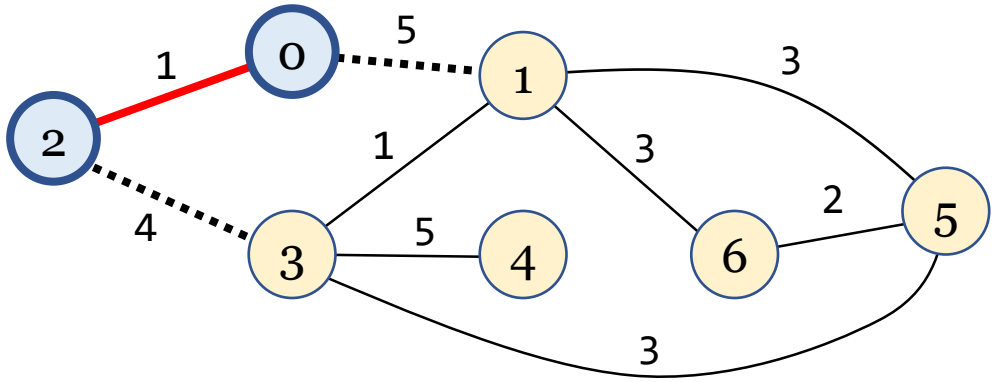
$parent[v]$: 정점 v와 최소 가중치로 연결되는 스패닝 트리에 있는 정점 u를 저장



0번 정점에서 시작

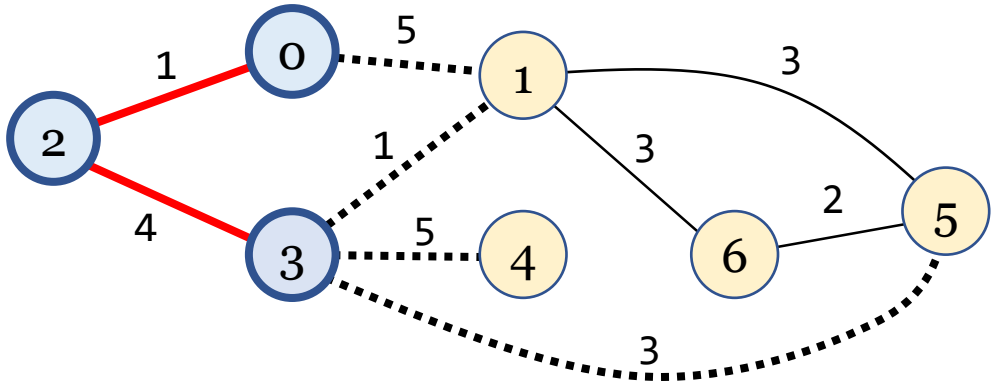
	0	1	2	3	4	5	6
$parent =$	0	0	0	−1	−1	−1	−1
$minweight =$	0	5	1	∞	∞	∞	∞

2번 정점을 선택



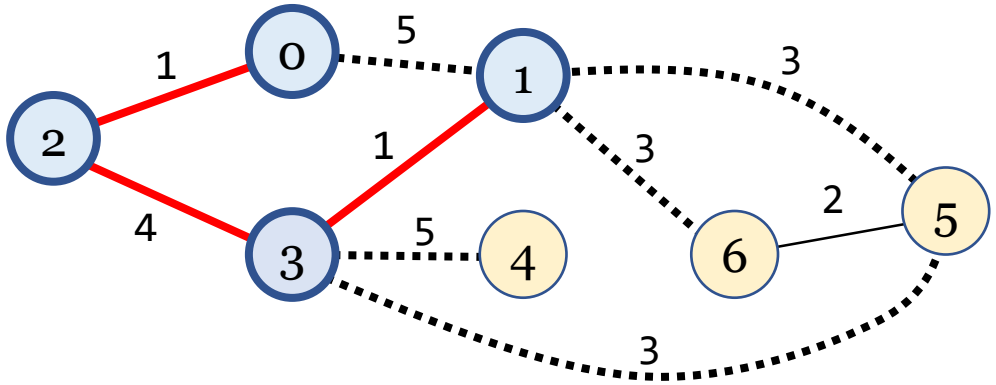
$V =$	0	1	2	3	4	5	6
$parent =$	0	0	0	2	−1	−1	−1
$minweight =$	0	5	1	4	∞	∞	∞

3번 정점을 선택



$V =$	0	1	2	3	4	5	6
$parent =$	0	3	0	2	3	3	−1
$minweight =$	0	1	1	4	5	3	∞

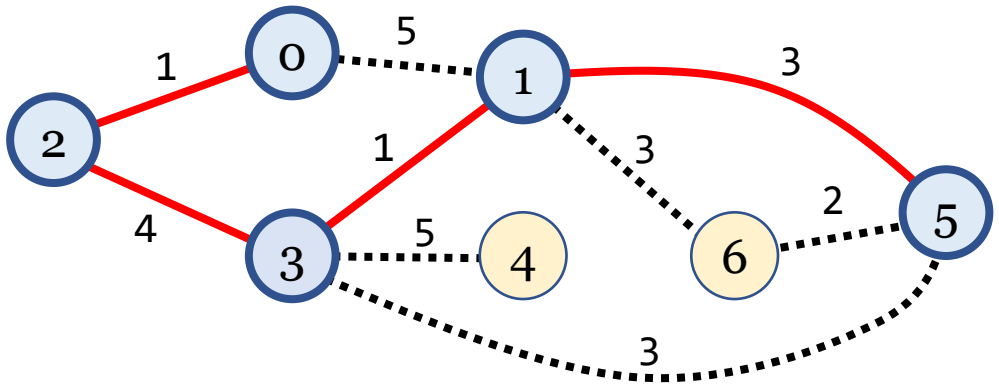
1번 정점을 선택



$V =$	0	1	2	3	4	5	6
$parent =$	0	3	0	2	3	3	1
$minweight =$	0	1	1	4	5	3	3

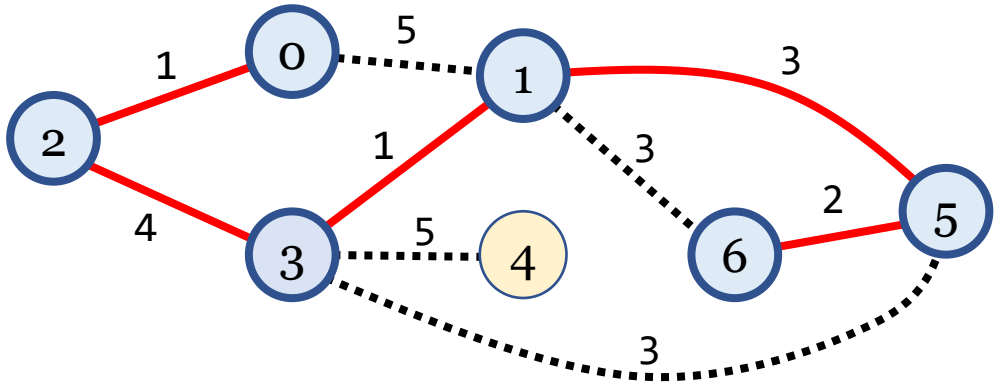
tie: 아무거나 선택해도 무방함

5번 정점을 선택



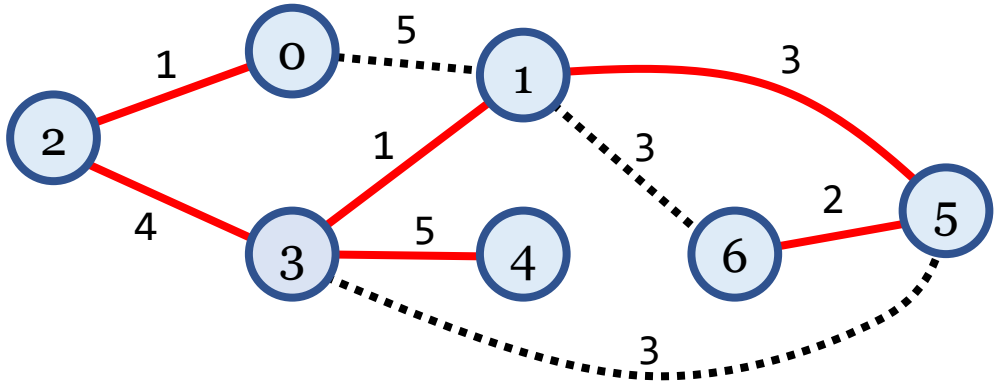
$V =$	0	1	2	3	4	5	6
$parent =$	0	3	0	2	3	3	5
$minweight =$	0	1	1	4	5	3	2

6번 정점을 선택

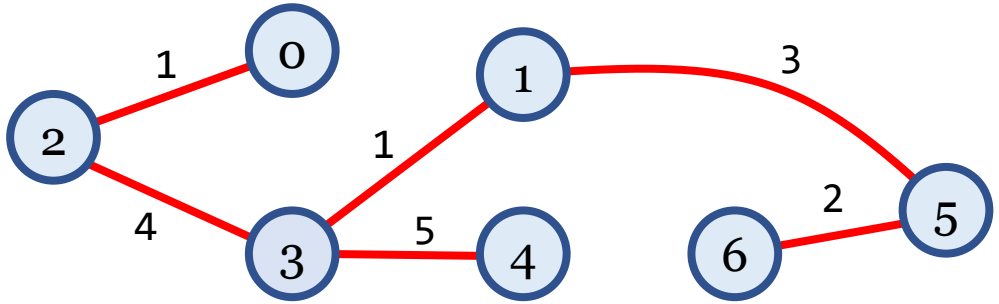


$V =$	0	1	2	3	4	5	6
$parent =$	0	3	0	2	3	3	5
$minweight =$	0	1	1	4	5	3	2

4번 정점을 선택



$V =$	0	1	2	3	4	5	6
$parent =$	0	3	0	2	3	3	5
$minweight =$	0	1	1	4	5	3	2



$0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 5 \rightarrow 6 \rightarrow 4$

$selected = \{(0,2),(2,3),(3,1),(1,5),(5,6),(3,4)\}$

$mincost = 1 + 4 + 1 + 3 + 2 + 5 = 16$

```
7 9
0 1 5
0 2 1
1 3 1
1 6 3
1 5 3
2 3 4
3 4 5
3 5 3
5 6 2
```

```python
# Prim 알고리즘 테스트용 코드
n, m = map(int, input().split())
adj = [{} for _ in range(n)]
for _ in range(m):
    u, v, w = map(int, input().split())
    adj[u][v] = adj[v][u] = w
cost, edges = prim(n, adj)
print(cost)
print(*edges)
```

```python
# MST 문제를 해결하는 Prim 알고리즘의 구현
def prim(n, adj):
    selected = [] # 추가된 간선을 저장한다.
    added = [False] * n # 해당 정점이 포함되었나?
    min_weight = [INF] * n # 트리에 인접한 간선 중 해당 정점에 닿는 최소 간선의 가중치
    parent = [-1] * n # 트리에 인접한 간선 중 해당 정점에 닿는 최소 간선의 정점
    min_weight[0] = parent[0] = 0 # 0번 정점을 시작점으로 한다.
```

```
    ret = 0 # 간선의 가중치의 합
    for _ in range(n):
        # 현재 선택된 노드 중 최소 가중치를 가지는 간선의 노드를 선택
        u = -1
        for v in range(n):
            if not added[v] and (u == -1 or min_weight[u] > min_weight[v]):
                u = v
        # 해당 간선을 추가하고, 가중치를 합산
        if parent[u] != u: # 출발 정점이 아니면
            selected.append((parent[u], u))
        ret += min_weight[u]
        added[u] = True
        # 선택된 노드와 인접한 노드들의 가중치 업데이트
        for v, weight in adj[u].items():
            if not added[v] and min_weight[v] > weight:
                min_weight[v] = weight
                parent[v] = u
    return ret, selected
```

Prim's 알고리즘: 우선순위 큐의 활용

우선순위 큐를 활용하지 않은 Prim 알고리즘은

추가할 새 정점을 찾는 작업을 O(n)에 할 수 있고,

모든 간선 (u, v)는 u를 트리에 추가할 때 한 번, v를 트리에 추가할 때 한 번만 검사하므로,

전체 시간 복잡도는 $O(|V|^2 + |E|)$

우선순위 큐를 활용하여 추가할 새 정점을 찾도록 하면?

우선순위 큐에 간선의 가중치와 해당 간선으로 연결하는 정점의 번호를 추가하고,

아직 방문하지 않은 정점 중 스패닝 트리로부터 거리가 가장 가까운 정점을 찾도록 한다.

전체 시간 복잡도는 $O(|E| \lg |V|)$

```
# 우선순위 큐를 사용하는 Prim 알고리즘
def prim(n, adj):
    selected = []; added = [False] * n
    PQ = [(0, 0, -1)] # (가중치=0, 출발정점=0, 부모=-1)로 초기화
    ret = 0 # 간선의 가중치의 합
    while PQ:
        w, u, p = heappop(PQ) # 우선순위 큐에서 트리에 가장 가까운 정점 정보 꺼냄
        if not added[u]:
            added[u] = True
            if p != -1: # 출발 정점을 제외한 간선 추가
                selected.append((p, u))
            ret += w
            for v, weight in adj[u].items():
                if not added[v]: # 모든 방문하지 않은 이웃 노드의 간선 정보를
                    heappush(PQ, (weight, v, u)) # 우선 순위 큐에 추가
    return ret, selected
```

근거리 네트워크 문제로 돌아와서:

주어진 건물 정보로부터 최소 스패닝 트리를 구하면서, 추가로 설치할 케이블 길이의 합을 구한다.

그래프의 구조: 정점: 건물, 간선: 건물과 건물 사이의 케이블

가중치가 있는 무방향 그래프: 가중치는 건물 좌표 간의 유클리드 거리

완전 그래프(*complete graph*): 모든 건물 사이에 케이블을 설치할 수 있으므로

따라서, Dense Graph에 유리한 Prim 알고리즘으로 문제를 해결한다.

이미 두 건물이 연결되어 있으면?

연결된 건물 사이에는 케이블을 설치할 필요가 없으므로 가중치를 0으로 두면 된다.

```
'''
P.6.3. 근거리 네트워크 (ID: LAN)
'''
for _ in range(int(input())):
    n, m = map(int, input().split()) # 건물의 수, 이미 설치된 케이블의 수
    x = list(map(int, input().split())) # 각 건물의 x 좌표
    y = list(map(int, input().split())) # 각 건물의 y 좌표
    W = [[0] * n for _ in range(n)] # 인접 행렬
    for u in range(n):
        for v in range(u, n):
            # 인접 행렬의 가중치: 연산 속도를 고려해 d^2으로 저장
            W[u][v] = W[v][u] = (x[u] - x[v])**2 + (y[u] - y[v])**2
    for _ in range(m):
        u, v = map(int, input().split())
        W[u][v] = W[v][u] = 0 # 이미 설치된 케이블의 가중치를 0으로 설정한다.
    print(f"{lan(n, W):08f}")
```

```python
# 근거리 네트워크 문제를 Prim 알고리즘으로 해결
def lan(n, W):
    dist = [W[0][i] for i in range(n)] # 출발 정점은 0번 정점
    ret = 0
    for _ in range(n - 1):
        # 현재까지 방문한 정점 집합에서 가장 가까운 정점을 찾는다.
        min = float('inf')
        for u in range(1, n):
            if 0 <= dist[u] < min:
                min = dist[u]
                vnear = u
        # 찾은 정점을 방문한다.
        ret += math.sqrt(dist[vnear])
        dist[vnear] = -1
        # 현재까지 방문한 정점 집합과의 거리를 업데이트한다.
        for u in range(1, n):
            if dist[u] > W[u][vnear]:
                dist[u] = W[u][vnear]
    return ret
```

Any Questions?

